

DAO

Desarrollo de Aplicaciones con Objetos

Python • POO • Persistencia • Interfaces • Patrones

UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ciclo Lectivo 2026



UTN
FRC

UNIVERSIDAD
TECNOLÓGICA NACIONAL
FACULTAD REGIONAL
CÓRDOBA

{ }

```
class DAO:  
    def aprender():  
        return software
```

Plantilla institucional de clases

Versión 2026

Lambdas: Comprensiones, Map, Filter, Reduce

(y otros animales fantásticos)



La clase de hoy

01

Programación Funcional

Paradigma de desarrollo

- Definición
- Ejemplo

02

Comprensiones

Estructuras de datos eficientes

- Definición de Comprensión en Python
- Sintaxis de colecciones optimizadas
- Ejemplos y casos de uso prácticos

03

Funciones Lambda

Sintaxis declarativa

- Definición de funciones anónimas
- Ventajas y simplificación de código
- Ejemplos de uso rápido

04

Orden Superior

Procesamiento de datos

- Función Map
- Función Filter
- Función Reduce

Programación Funcional: Definición

¿Qué es la programación funcional?

Es un paradigma de programación donde el desarrollo se estructura mediante la evaluación de **funciones matemáticas**, evitando el uso de estados mutables y datos que cambian en el tiempo.

Principios fundamentales:

- **Funciones Puras:** El mismo valor de entrada siempre retorna idéntico resultado sin efectos secundarios.
- **Inmutabilidad:** Una vez creada una estructura de datos, esta no se puede modificar.
- **Funciones de Primer Orden:** Pueden tratarse como variables (pasarse como argumentos o retornarse).

Ejemplo comparativo en Python

```
1 print ("Hola mundo")
2
3 imprimir = print
4
5 imprimir ("Hola mundo desde funcional")
6
```

Comprensión: Definición

¿Qué es una comprensión?

Una “**comprensión**” es un método abreviado y compacto que permite crear listas u otras colecciones en Python de forma eficiente.

Características clave

- **Sintaxis particular:** Ofrece una forma más clara y legible de escribir código.
- **Mayor eficiencia:** En muchos casos, es más rápida y óptima que los métodos tradicionales.
- **Alternativa elegante:** No es la única opción para crear listas, pero es la preferida en el desarrollo moderno.

Sintaxis en Python

```
mi_lista = [<expresion> for <item> in <iterable> if <condicion>]
```

Comprensión: Explicación

```
mi_lista = [<expresion> for <item> in <iterable> if <condicion>]
```

expresion

Qué quiero hacer

Define el valor o la transformación que se aplicará a cada elemento de la nueva lista.

item

Sobre qué lo quiero hacer

Representa la variable temporal que toma el valor de cada elemento en la iteración.

iterable

De dónde saco el item

La colección de origen (lista, tupla, rango, etc.) que se va a recorrer.

condición (opcional)

Si debe cumplir algún requisito

Filtro opcional para incluir solo los elementos que cumplan con este criterio específico.

Comprensión: Ejemplo

```
mi_lista = [1,2,3,4,5,6,7,8,9,10]

pares = [nro for nro in mi_lista if nro%2 == 0]
sig1 = [nro+1 for nro in mi_lista if nro%2 == 0]
sig2 = [nro+1 for nro in pares]

sig3 = []
for nro in mi_lista:
    if nro%2 == 0:
        sig3.append(nro+1)
```

Comprensión: Diccionario

```
mi_dict = [<clave>:<valor> for <item> in <iterable> if <condicion>]
```

clave**Criterio único**

El criterio único que identifica cada elemento dentro de la estructura del diccionario.

valor**Dato asociado**

El contenido o dato específico que se almacena y asocia directamente con la clave.

item**Elemento**

Cada uno de los componentes individuales de la lista o colección recorrida.

iterable**Origen de datos**

La colección de origen (lista, rango, etc.) desde donde se obtienen los datos.

condición (opc.)**Filtro opcional**

Permite filtrar y procesar los elementos de acuerdo a un criterio específico definido.

Comprensión: Ejemplo en diccionario

```
# Obtener aquellas entradas que tengan Data en su titulo  
# Metodo iterativo  
data1 = {}  
for p in persons:  
    if 'Data' in p['title']:  
        data1[p['name']] = p['title']  
print ("Método iterativo:",data1)  
# Metodo por comprension  
data2 = {p['name']:p['title'] for p in persons if 'Data' in p['title']}  
print ("Por comprension:",data2)
```

Funciones lambda: Definición

¿Qué es una función lambda?

Es una función que no tiene nombre. Es una alternativa compacta y potente para estructurar comportamientos de manera inmediata dentro del código.

- **Paradigma Funcional:** Asociada fuertemente a la programación funcional para permitir un estilo declarativo.
- **Ejecución Simple:** Sirve para ejecutar pequeñas porciones de código de manera simple y directa.
- **Sintaxis en una línea:** Ideal para ser pasada como argumento a funciones de orden superior como map, filter o reduce.

Funciones lambda: Explicación

```
funcion = lambda argumento : expresión
```

funcion**Nombre de función**

Nombre asignado a la variable que contendrá la función lambda para poder invocarla.

lambda**Palabra clave**

Palabra clave requerida de Python, se escribe tal cual para definir la función anónima.

argumento**Parámetros**

Representa el valor o valores de entrada que se envían a la función para ser procesados.

expresión**Lógica interna**

El código que se debe ejecutar para el argumento, el cual siempre debe devolver un valor.

Funciones lambda: Ejemplo simple

```
cuadrado = lambda x : x**2

print (cuadrado(3))

cuadrado_mayor_diez = lambda x : True if x**2 > 10 else False

print (cuadrado_mayor_diez(2))
print (cuadrado_mayor_diez(5))
```

Map, Filter y Reduce

Estos métodos de orden superior sirven para procesar listas de forma compacta y hacen un uso intensivo de funciones lambda.

map()

Transformación de elementos

- Aplica una función a cada elemento de una lista de entrada.
- Devuelve una nueva lista (o iterable) con los valores resultantes de dicha función.

filter()

Selección condicional

- Aplica una función de filtro a cada elemento de la colección.
- Devuelve una lista únicamente con los elementos que evalúan a **True**.

reduce()

Reducción acumulativa

- Aplica la función de forma sucesiva sobre los elementos.
- Devuelve un **único resultado acumulado** (no una lista con múltiples elementos).

Map: Ejemplo de uso

¿Cómo funciona map()?

Aplica una función determinada a cada uno de los elementos de un iterable (como una lista) y retorna un nuevo iterador con los resultados transformados.

Aspectos clave:

- **Sintaxis:** map(función, iterable)
- **Transformación limpia:** Evita bucles explícitos, promoviendo un código más declarativo.
- **Retorno:** Devuelve un objeto map que puede convertirse fácilmente en una lista.

Ejemplo de uso en Python

```
from rich import print

lista = [1, 2, 3, 4, 5]

cuadrados = list(
    map(
        lambda x :
            x**2,
        lista)
)

print ("Lista:", lista)
print ("Cuadrados:", cuadrados)
```

Filter: Ejemplo de uso

La función filter()

Permite filtrar los elementos de una colección para los cuales una función condicional retorna verdadero.

Sintaxis:

filter(funcion, lista)

- **funcion:** Condición (retorna True o False). Comúnmente una expresión lambda.
- **lista:** La colección o iterable cuyos elementos serán evaluados.

Código de ejemplo

```
lista = list(range(-5, 5))
menor_cero = list(
    filter(
        lambda x:
            x < 0,
        lista)
    )
print("Lista: ", lista)
print("<0:", menor_cero)
```

Reduce: Ejemplo de uso

`reduce(función, lista)`

Aplica una función de forma acumulativa a los elementos de una colección, de izquierda a derecha, para reducirla a un solo valor.

Aspectos clave del ejemplo:

- **Importación requerida:** Se debe importar desde el módulo `functools`.
- **Función Lambda:** Usa `lambda x, y: x+y` para acumular.
- **Resultado:** La lista `[1, 2, 3, 4, 5]` se procesa obteniendo **15**.

Código de ejecución en Python

```
lista = [1, 2, 3, 4, 5]

# Para usar reduce hace falta importar el módulo functools
sumatoria = reduce(
    lambda x,y :
        x+y,
    lista)
print ("Lista:", lista)
print ("Sumatoria:", sumatoria)
```

Material Adicional



Material Adicional



Comprensión de listas y otras colecciones

<https://recursospython.com/guias-y-manuales/compression-de-listas-y-otras-colecciones/>



Comprensión de listas. Qué son y como funcionan

<https://j2logo.com/list-comprehensions-en-python/>



Funciones lambda

<https://www.freecodecamp.org/espanol/news/expresiones-lambda-en-python/>

Material Adicional



Map, Filter, Reduce

https://python-intermedio.readthedocs.io/es/latest/map_filter.html



Lambda + Map/Filter/Reduce

<https://j2logo.com/python/funciones-lambda-en-python/>



Map Filter and Reduce (Inglés)

https://www.learnpython.org/en/Map%2C_Filter%2C_Reduce



Map,Filter,Reduce with examples (inglés)

<https://stackabuse.com/map-filter-and-reduce-in-python-with-examples/>

Ejercicios Prácticos Integradores

Comprensión

Optimización de listas

Creación y filtrado rápido de colecciones en una sola línea de código.

- Listas y Diccionarios
- Uso de condicionales

Map

Transformación

Aplicación de una función sobre cada elemento de un iterable.

- Retorno de generadores
- Mapeo con Lambdas

Filter

Filtrado lógico

Selección de elementos que cumplen una condición lógica específica.

- Funciones de descarte
- Evaluación booleana

Reduce

Reducción acumulativa

Reducción de una colección a un único valor acumulado.

- Módulo functools
- Estados acumulados

Comprensión, Map, Filter, Reduce

Punto de partida

Dada la siguiente lista de strings en Python, realiza las actividades propuestas utilizando comprensiones y funciones de orden superior:

```
# Lista a procesar  
lst = ["Aruba", "Jamaica", "Bermuda",  
       "Bahama", "Key Largo", "Montigo"]
```

01

Primer carácter de cada palabra

Crear una lista por comprensión que contenga el primer carácter de cada palabra.

02

Longitud de cada palabra

Crear una lista por comprensión que contenga la longitud de cada palabra.

03

Cantidad total de caracteres (Reduce)

Utilizando Reduce, calcular la cantidad total de caracteres que contienen todas las palabras de la lista lst.

Comprensión, Map, Filter, Reduce

01 . COMPRENSIÓN

Crear lista

Utilizando las rutinas de `random` (`import random`), crear una lista por comprensión.

La lista debe contener **50 números enteros** al azar, en el rango de **1 a 100**.

02 . FILTER

Filtrar elementos

Utilizando la función `Filter`, separar los números generados en dos colecciones:

- Una lista con los mayores a **50**.
- Otra lista con los menores o iguales a **50**.

03 . MAP

Transformar datos

Utilizando la función `Map`, procesar la lista inicial:

Obtener una nueva lista que contenga solo los **números impares** pero con su **signo invertido** (negativos).

04 . REDUCE

Reducir a un valor

Utilizando la función `Reduce`, realizar la consolidación:

Obtener la **sumatoria total** de la lista que fue generada en el paso anterior.

¡Gracias!

Hasta la próxima



**UTN
FRC**

UNIVERSIDAD
TECNOLÓGICA NACIONAL
**FACULTAD REGIONAL
CÓRDOBA**